HW-05

IP Routing – Longest Prefix Match

1 Overview

This exercise provides hands-on experience with IP routing of packet in a network and use of longest prefix match in IP routing.

2 Learning Objectives

- Understand working of IP protocol.
- Understand and implement IP Routing.
- Understand and implement Longest Prefix Match in IP Routing.

3 Learning Resources

1. Kurose, Ross: "Computer Networking: 8th edition – A Top Down Approach", Chapter 02.

4 Environment

This assignment can be carried out in a docker based environment using Docker Desktop, which is an application environment for your laptop environment that enables running of containerized applications.

5 Network Setup

5.1 Network Topology

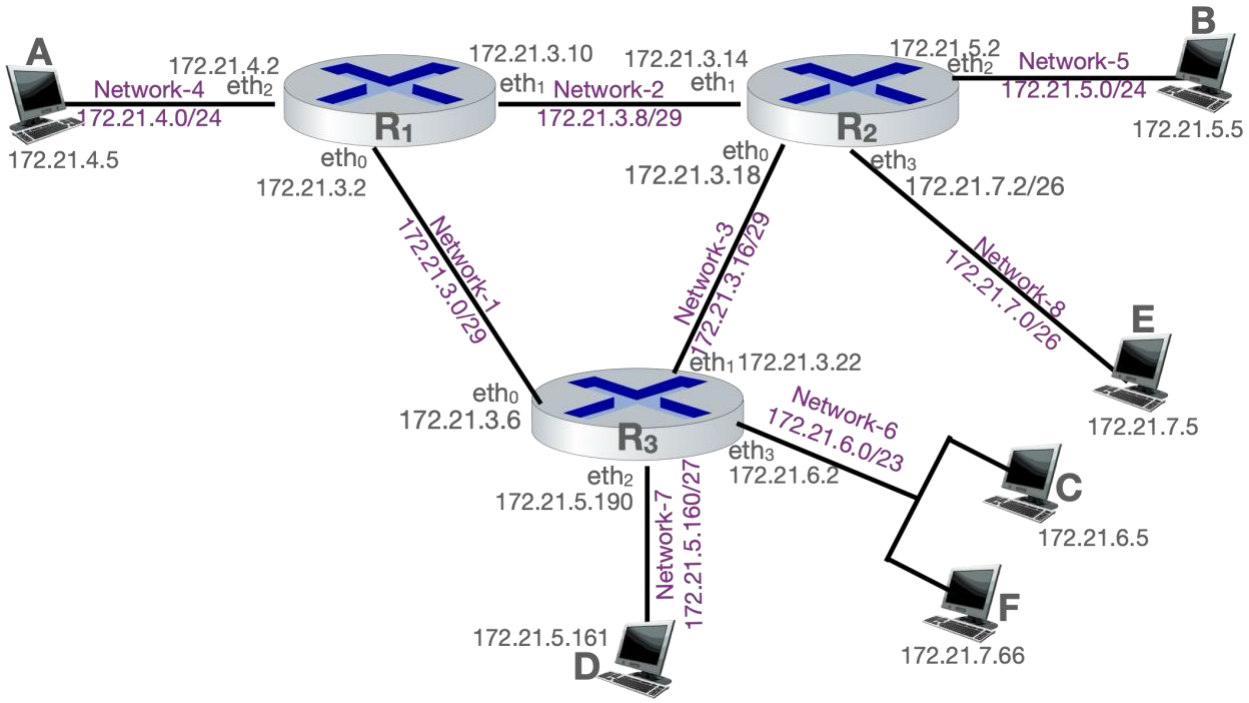

Figure 1: network setup

To study and understand IP routing and Longest Prefix Match, create the network (as shown in Figure 1). The network consists of 3 routers and 6 hosts connecting 8 networks. The IP addresses shown are for illustration and should be changed as applicable. The addresses shown in the diagram belongs to network ranges (where X=3, Y=X+3), `172.21.X.0/29` (network-1), `172.21.X.8/29` (network-2), `172.21.X.16/29` (network-3), `172.21.(X+1).0/24` (network-4), `172.21.(X+2).0/24` (network-5), `172.21.Y.0/23` (network-6), `172.21.(X+2).160/27` (network-7), and `172.21.(Y+1).0/26` (network-8). For your network, choose the value of `X` as last 2 digits of UMBC Id, and if X is odd choose `Y=X+3`, otherwise (when `X` is even), choose `Y=X+4`. This is because with mask of `/23`, Y should be an even number. Create your network diagram as per values applicable to you and modify these IP address values in provided docker compose sample file `LPM-Routing.yml`. Having a diagram with applicable values will be handy when implementing the network and diagnosing network configuration and implementation.

Note:

- The docker images have been updated, so you are advised to remove the existing images from your laptop cache as shown below. Thus, when creating docker instances, it will automatically fetch the last updated image.
```
yml> git pull
yml> docker-compose -f LPM-Routing.yml build --no-cache R1 HA
```
- The IP addresses of host D, Host E and `eth2`/`eth3` interface of Router R2/R3 are assigned for place holder purposes. The network-5 overlaps network-7, and network-6 overlaps network-8. The docker desktop does not permit creation of overlapping network in the compose file and these are to be assigned manually after the network is up.
- In normal network setup, the network-1, network-2 and network-3 require only two assignable addresses and thus /30 netmask should be sufficient. But when docker instances are created, it uses the first assignable address as default gateway, and thus we need 3 assignable addresses. Hence these networks are created with /29 netmask.

5.1.1 Verification of Network Connectivity

For hosts HA, HB, HC and HF, verify connectivity/reachability to *their connected router*. This can be done by sending ping packets and observing their responses.

6 Assignment Description: Longest Prefix Match

6.1 Assigning IP addresses belonging to smaller overlapping subnetwork

The provided docker compose example file does not have network entries for overlapping smaller subnets 172.21.5.160/27 and 172.21.7.0/26. The former is contained in (fully overlapped by) 172.21.5.0/24 and latter is contained in (fully overlapped by) 172.21.6.0/23. This is because docker compose file does not permit overlapping networks, i.e., it gives error when container instances are created with overlapping networks. Thus, a workaround mechanism has been used for this assignment to create overlapping entries. A filler network entry 172.21.(X+2).199 is used for host D and 172.21.(x+3).199 for host E. Similarly, the address 172.21.(X+2).99 is used for

router R2 and 172.21.(X+3).99 for R3, corresponding to overlapping networks. This workaround mechanism ensures that corresponding network interface is created. After respective network interfaces are created, we need to assign the appropriate IP addresses belonging to these overlapping networks as below:

6.1.1 Router R2

Login to R2 container and note down ethernet interface corresponding to IP address 172.21.(X+2).99. Assign the IP address belonging to smaller subnet 172.21.(Y+1).0/26 contained within 172.21.Y.0/23 to this interface (e.g., `eth3`). Use the second assignable address within this subnet, i.e., 172.21.(Y+1).2 in this network to the network interface `eth3` as below. If your interface name is not `eth3`, then use corresponding interface instead of `eth3`.

```
$ docker exec -it R2 bash
root@R2:/# ip addr flush dev eth3
root@R2:/# ip addr add 172.21.(Y+1).2/26 dev eth3
root@R2:/# ip link set dev eth3 up
```

6.1.2 Router R3

Find out the ethernet interface (e.g., `eth2`) corresponding to filler network 172.21.(X+2).99 and use the last assignable address in the smaller subnet of 172.21.(X+2).160/27, i.e., the IP address 172.21.(X+2).190 to this interface, e.g., as below:

```
$ docker exec -it R3 bash
root@R3:/# ip addr flush dev eth2
root@R3:/# ip addr add 172.21.(X+2).190/27 dev eth2
root@R3:/# ip link set dev eth2 up
```

6.1.3 Host D

Using the process above, use the first assignable IP address to its interface (corresponding to filler value), e.g., `eth0` belonging to smaller subnet of 172.21.(X+2).160/27, i.e., assign the IP address 172.21.(X+2).161/27

6.1.4 Host E

Using the process above, assign an IP address to interface (corresponding to filler value), e.g., `eth0`, belonging to smaller subnet 172.21.(Y+1).0/26. To be consistent with address assignment for other hosts (e.g., A, B, C), assign the IP address 172.21.(Y+1).5/26 to the interface `eth0`.

6.2 Configure Forwarding Tables for Routers

From the docker compose file, the forwarding tables contains entries only for directly connected networks and there are no entries in the forwarding table for any other networks. For example, forwarding table for each router contains three entries for each of the 3 directly connected network as shown below:

```
$> docker exec R1 ip route
default via 172.21.3.14 dev eth1
172.21.3.0/29 dev eth0 proto kernel scope link src 172.21.3.2
172.21.3.8/29 dev eth1 proto kernel scope link src 172.21.3.10
```

```
172.21.4.0/24 dev eth2 proto kernel scope link src 172.21.4.2
$>
$> docker exec R2 ip route
default via 172.21.3.22 dev eth0
172.21.3.8/29 dev eth1 proto kernel scope link src 172.21.3.14
172.21.3.16/29 dev eth0 proto kernel scope link src 172.21.3.18
172.21.5.0/24 dev eth2 proto kernel scope link src 172.21.5.2
172.21.7.0/26 dev eth3 proto kernel scope link src 172.21.7.2
$>
$> docker exec R3 ip route
default via 172.21.3.2 dev eth0
172.21.3.0/29 dev eth0 proto kernel scope link src 172.21.3.6
172.21.3.16/29 dev eth1 proto kernel scope link src 172.21.3.22
172.21.5.160/27 dev eth2 proto kernel scope link src 172.21.5.190
172.21.6.0/23 dev eth3 proto kernel scope link src 172.21.6.2
$>
```

That means when any of the router receives a packet with an IP address that does not belong to any of its directly connected interfaces, it will not be able to route the packet. To enable proper connectivity, you need to define routing entries. An example is given below to create routing entries for router `R3` so as to reach all other network.

```
root@R3:/# ip route add 172.21.4.0/24 via 172.21.3.2
root@R3:/# ip route add 172.21.5.0/24 via 172.21.3.18
root@R3:/# ip route add 172.21.7.0/26 via 172.21.3.18
```

Please note that second entry above also overlaps (fully contains) its directly connected network 172.21.5.160/27. Thus, if router `R3` receives a packet with destination IP address as 172.21.5.170, it will match two entries in the forwarding table, one corresponding to 2nd entry given above for network 172.21.5.0/24 and other one corresponding to its directly connected network 172.21.5.160/27. But since latter has longer prefix of /27, it will forward the packet on the interface $\texttt{eth}_2$ (directly connecting the network 172.21.5.160/27). But, if this router R3 gets a packet with destination IP Address 172.21.5.193, it would send the packet to router R2 for further processing.

Using your network diagram, create appropriate routing entries for Routers `R1` and `R2`.

6.3 Configure Forwarding Tables for Hosts

Using the similar consideration and considering your network setup, create routing entries in router R1 and R2. Similarly, create routing entries in each of the host HA, HB, HC, HD, HE and HF. For the hosts, you can create one simply default route entry since it has just one IP address and all of its packets must be sent to its directly connected router. For example, default entry for host A can be added as follows to send to its default router with IP 172.21.4.2.

```
root@HA:/# ip route add default via 172.21.4.2
```

Using your network diagram, similarly create appropriate default routing entries for Hosts HB, HC, HD, HE and HF.

6.3.1 Forwarding Table for host HB and HC

These hosts belong to larger network whose smaller subnet is elsewhere. For example, for HB (172.21.5.0/24), there is smaller overlapping subnet 172.21.5.160/27 which is connected to Router R3. Thus, host HB needs specific forwarding entry for this overlapping network. Similarly, host HC would need specific forwarding entry for overlapping sub network 172.21.7.0/26.

Forwarding table entry at host HB would be made by

```
root@HB:/# ip route add 172.21.5.160/27 via 172.21.5.2
```

Forwarding table entry at host HC and host HF would be made by

```
# ip route add 172.21.7.0/26 via 172.21.6.2
```

6.4 Study and Analysis of Longest Prefix Match

After creating all the required routing entries, check that you are able to reach each host from every other host. To check reachability, just use ping command, e.g., from host HA (172.21.4.5) use the command to send 2 ping packets to host B (172.21.5.5) as below and it should return success response.

```
$docker exec -it HA ping -c2 172.21.5.5
```

6.5 Study and Analysis of Routing Loop

In each of the router (R1, R2, R3) create one route entry for the network 172.21.200.0/24 forwarding the packet to its next router i.e.,

a) R1 should forward any packet with destination address in 172.21.200.0/24 to R2,
b) R2 should forward any packet with destination address in 172.21.200.0/24 to R3
c) R3 should forward any packet with destination address in 172.21.200.0/24 to R1

Thus, when any host sends a packet to some destination in the network 172.21.200.0/24, for example, ping to address 172.21.200.200 should result in ICMP error `Time to live exceeded`.

6.6 Study and Analysis of Unreachable Network

Since there is no default route entry in any router, thus, when any host sends a packet to some destination in a unrouted network. e.g., ping to address 1.2.3.4 should result in `Destination Net Unreachable`.

7 Assessment

7.1 A Readme.txt file providing following

a. UMBC Id, Name.
b. Summary of your learning
c. Challenges faced in doing the assignment and how did you address it.

d. Approximate time spend in completing the assignment.

7.2 IP addresses of all interfaces

IP address of all interfaces of each docker container i.e., output of following commands for each router (R1, R2 and R3) and each host (HA, HB, HC, HD. HE and HF) i.e.,

```
$ docker exec -it R1 ip -4 -br addr
$ docker exec -it R2 ip -4 -br addr
$ docker exec -it R3 ip -4 -br addr
$ docker exec -it HA ip -4 -br addr
$ docker exec -it HB ip -4 -br addr
$ docker exec -it HC ip -4 -br addr
$ docker exec -it HD ip -4 -br addr
$ docker exec -it HE ip -4 -br addr
$ docker exec -it HF ip -4 -br addr
```

7.3 Routing table of Routers

Forwarding table of each router i.e., output of following commands for each router (R1, R2 and R3).

```
$ docker exec -it R1 ip route
$ docker exec -it R2 ip route
$ docker exec -it R3 ip route
```

7.4 Network Reachability

Show that each host is reachable from each other i.e., output of following commands

```
$ docker exec -it HA ping -c2 <IP address of HD>
$ docker exec -it HA ping -c2 <IP address of HE>
$ docker exec -it HA ping -c2 <IP address of HF>
$ docker exec -it HB ping -c2 <IP address of HD>
$ docker exec -it HB ping -c2 <IP address of HE>
$ docker exec -it HB ping -c2 <IP address of HF>
$ docker exec -it HC ping -c2 <IP address of HD>
$ docker exec -it HC ping -c2 <IP address of HE>
$ docker exec -it HC ping -c2 <IP address of HF>
$ docker exec -it HD ping -c2 <IP address of HE>
$ docker exec -it HD ping -c2 <IP address of HF>
$ docker exec -it HE ping -c2 <IP address of HF>
```

7.5 Routing Loop

Show that routing loop exists for network 172.21.200.0/24 i.e., ping to address 172.21.200.200 should result in ICMP Error `Time to live exceeded`. Thus, show the output of following commands

```
$ docker exec -it HA ping -c2 172.21.200.200
$ docker exec -it HB ping -c2 172.21.200.200
$ docker exec -it HC ping -c2 172.21.200.200
$ docker exec -it HD ping -c2 172.21.200.200
```

```
$ docker exec -it HE ping -c2 172.21.200.200
$ docker exec -it HF ping -c2 172.21.200.200
```

7.6 Unrouted Network

Show that any unknown network is unreachable, i.e., ping to address 1.2.3.4 should result in ICMP Error `Destination Net Unreachable`. Thus, show the output of following commands

```
$ docker exec -it HA ping -c2 172.21.200.200
$ docker exec -it HB ping -c2 172.21.200.200
$ docker exec -it HC ping -c2 172.21.200.200
$ docker exec -it HD ping -c2 172.21.200.200
$ docker exec -it HE ping -c2 172.21.200.200
$ docker exec -it HF ping -c2 172.21.200.200
```

8 Evaluation:

Total marks for assignment: 15 marks

i. 1 mark for diagram showing your actual network with IP addresses as applicable to you. This diagram can be hand-drawn and photo taken or keynote/powerpoint slide. You can modify the attached ppt/keynote slide to draw your network diagram.
ii. 2 marks for assigning IP address in overlapping smaller subnetworks with subnet mask of /26 and /27 (section 7.2)
iii. 3 marks for configuring routing tables (section 7.3)
iv. 4 marks for showing reachability (section 7.4)
v. 3 marks for showing ICMP error `Time to live exceeded`. (section 7.5)
vi. 2 marks for showing ICMP error `Destination Net Unreachable`. (section 7.6).

9 Note

Any plagiarism activity will result in penalties of being awarded 0 marks.